n = 83: the linear run finder in the rank stage's probe loop, exact change only (Tier 1 design)

2026-10-21

n = 83: the linear run finder in the rank stage's probe loop, exact change only (Tier 1 design)

Status (2026-10-21): design frozen, run launched, no result yet. This experiment carries round 2's proven run-finder canonicalizer (C7) from the root-index build into the **rank stage's probe loop**. It is an exact change only: every probe key, probe count, relation, root call, counter and rank table stays the parent's. Every profile figure and saving below comes from the shared cloud-agent VM. They are local diagnostics, used only to decide whether a run is worth doing and to size it. The Run section records the trigger and the run URL. The result will be added in its own section, leaving the design text unchanged.

Why this run

The n = 83 a = 1 baseline row is the **round-2 index-build producer**:

- experiment `koblitz-n83-indexbuild2-20261020`;
- PR #1009, run 39464544962;
- source `4008a47a...`, binary `ic_online_indexbuild2_r770`;
- Tier 1-qualified at 1.1169 [1.1110, 1.1231] over the index-build row.

In that run the row (its side B) spent its 11.54 s end to end as follows. The rank stage is 36.5% of it.

Stage (parent run, side B means)	Time (ms)	Share of E2E
root-index build	5,170.8	45%
rank stage	**4,216.1**	**36.5%**
full scan-order self-test	1,436.9	12%
online (one target)	332.4	3%
rest of setup	about 381	3%

The parent's README names this follow-up. The rank stage's probe loop canonicalizes each key with lift_scan (`KIC_RANK_CANON=lift`), so C7 can carry over there exactly.

1. Profile of the rank stage (local diagnostic)

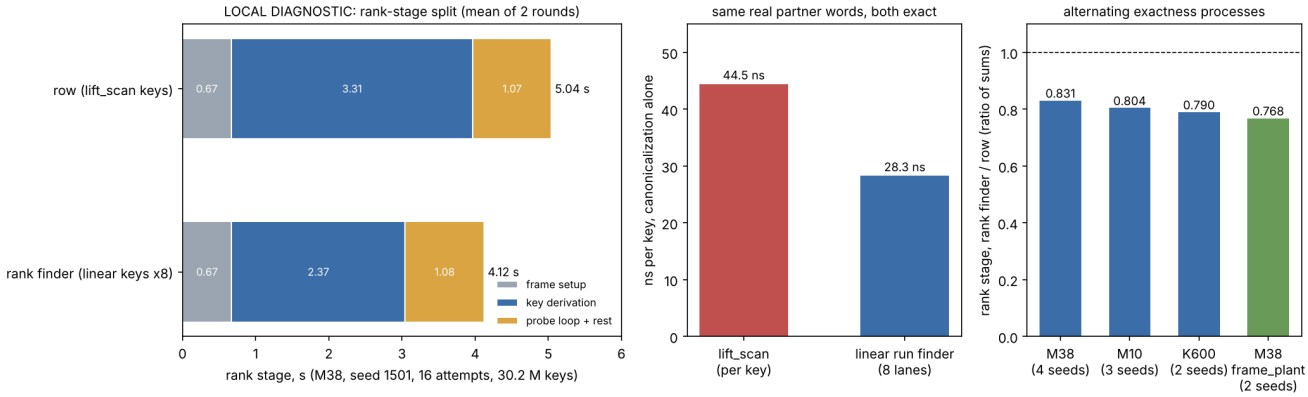


Figure 1: LOCAL DIAGNOSTIC. Left: the rank stage split into frame setup, key derivation and the probe loop, for the row and the rank finder (M38, rank seed 1501, 16 attempts, 30.2 M keys, mean of 2 rounds). Middle: the canonicalization alone on the same real partner words. Right: the rank-stage ratio per base in the alternating exactness processes. Rendered from `inputs/local_profile.json` by `make_figures.py` (svg, pdf).

Method. `profile/profile_patch.py` patches a *scratch copy* of each producer. These copies are never committed or built as producers. The patch adds two per-state timers inside `scan_state128_frame`, the rank stage's probe loop (`KIC_RANK_SOLVE=frame`). One timer covers the frame setup (frame roots, frame build, the batched inverse); the other covers key derivation (partner words, canonicalization and prefetch). The probe loop and the relation checks are the rest of the rank stage.

With `KIC_RANKPROF=canon` the patch also times the canonicalization alone, after each state's keys are complete. It runs on that state's real partner words, and its output is discarded through `black_box`. Two canonicalizers are timed this way: lift_scan per word, and the linear run finder eight words at a time.

`profile/run_profile.sh` runs each process with `arm.py`'s rank-arm environment on the M38 closure base, pinned with `taskset -c 3`, two rounds per mode. The build used rustc 1.97 (the workflow pins 1.99). `perf` is not available. `profile/summarize.py` writes `inputs/local_profile.json`.

Rank stage, M38 seed 1501 (30,197,392 keys, 90,956 states)	Row (ms)	Rank finder (ms)
frame setup (frame roots, frame build, batched inverse)	665	673
key derivation (partner words, keys, prefetch)	**3,306**	**2,371**
probe loop, relation checks and the rest	1,069	1,079
rank stage	**5,040**	**4,122**

Canonicalization alone, on the same partner words	ns per key
lift_scan (the row's call)	44.5
linear run finder, 8 lanes (round 2's `lift_linear_lanes8`)	28.3

Findings:

1. **Key derivation is two thirds of the rank stage** (3.31 of 5.04 s, about 110 ns per key). It includes about 44 ns of lift_scan per key.

2. **The saving is larger than the difference of the stand-alone costs.** Key derivation drops by 0.94 s, but the stand-alone costs differ by only 0.49 s (1.34 against 0.85 s). Interleaving eight lanes with the field arithmetic hides more of the canonicalization than the per-key serial chain did.
3. **What is left** is the field arithmetic per candidate:
   - 2 products and 2 `to_normal` maps;
   - the rotations;
   - the linear finder (about 27 ns per key).

   It is not touched here.

In the alternating exactness processes (section 4), the rank finder's rank stage over the row's was:

Base	Ratio	Seeds
M38	0.831	4
M10	0.804	3
K600	0.790	2
M38 with `frame_plant`	0.768	2

That is a measurable local saving (about 17% of the rank stage on M38), so a run is designed below.

2. The change and its proof

C7 itself (the linear run finder) is round 2's, unchanged. Its proof and tests are in the parent's README, section "C7: the linear run finder (proof)", and `make_indexbuild2_source.py` (verbatim here). In outline: for a nonconstant N-bit word v, let z_1 = !v and z_(j+1) = z_j & rotl(z_j, 1). Then bit p of z_j is set iff bits p, ..., p-j+1 are all zero (cyclically). So the last nonzero z_j is Z_L, the tops of all longest zero runs. That is exactly the run set lift_scan's greedy lifting ends with. The candidates (s = N - 1 - p) and the choice between them are lift_scan's own code: the least rotation, ties to the smaller shift, an order-independent minimum. The words 0 and all-ones give shift 0.

Round 2 tested it in four ways:

- exhaustively against the full scan on every word of 3..20 bits;
- exhaustively against lift_scan on eight two-limb geometries;
- on 2^22 random and about 207,000 structured 83-bit words;
- on every stored root of K600, M38 and M10.

What changes here. `scan_state128_frame` is used only by the rank stage (`KIC_RANK_SOLVE=frame` and `frame_plant`; the online target scan uses another loop). In the parent:

1. Each partner word's key is `target_canonical::<CANON_LIFT>`, i.e. `lift_scan_wide(v, 83)`, computed right after the word.
2. Then its home slot is prefetched.
3. Then the probe loop looks the keys up in order.

In the derived loop (`C == CANON_LIFT && n == 83`):

- the same partner words are pushed in the same order, with the same candidate index and the same pair starts;
- whenever eight keys are pending, their (canonical, shift) come from `lift_linear_lanes8::<83>` and their home slots are prefetched;
- the at most seven keys left at the end of a state take the parent's call;
- every key is complete before the first lookup;
- the probe loop, the exact-order resolution at a hit and the relation check are the parent's text.

For any other canonicalizer or field size, the parent's code runs.

So the key vector equals the parent's element by element, and the probe loop reads nothing else that changed. That makes the probe positions, first hit, relation, root calls and counters the parent's. The prefetch is a hint. `frame_plant` still recomputes every key with the rank-cut loop's lift_scan path and compares them key by key. In the derived binary, that means every linear key is checked against lift_scan.

A counted derivation. `make_rankfinder_source.py` follows the `make_*_source.py` pattern:

1. It re-derives the row's producer with the parent's `make_indexbuild2_source.py` (verbatim). That script re-derives `bc6a31c7...` through its verbatim chain and runs its own inverse check.
2. It requires the result to be `4008a47a...`.
3. It applies **6 counted replacements**, each of which must occur exactly once:
   - the header comment;
   - one constant (`RANK_PROBE_LANES = 8`);
   - the start of the key loop (`keys`);
   - the per-candidate push (`push`);
   - the summary field `rank_probe_pipeline` = `linear_canon_lanes8` (the parent emits no such field);
   - one appended test module.
4. `check_inverse()` undoes them on every derivation and must get the parent's bytes back.

The derived source is `301615b3...` (`koblitz_orbit_dlp_fast_online_rankfinder.rs`), and the binary is `ic_online_rankfinder_r770`.

`rankfinder_tests` (2 new tests). The parent's `scan_state128_frame` is kept there **verbatim** (re-indented, renamed `parent_scan_state128_frame`) as the reference.

- `derived_frame_loop_reproduces_the_parent_loop_on_every_state`. On every state of a small folded index (9 orbits, lift index), with 8 planted four-point targets, it compares the derived loop and the parent's. They must give identical:
  - relation (point indices, x codes, intermediates, probes);
  - probe count, root calls, filter passes and relation checks;
  - full key vectors (canonical, partner, shift, candidate) and pair starts.

  It covers `PIPE_FRAME`, `PIPE_FRAME_PLANT`, `PIPE_FRAME_SWAPPED` and the full-scan canonicalizer (the parent's path). It asserts that hits and the scalar tail both occur.
- `derived_frame_extract_equals_the_rankcut_extract`. On 40 planted targets it checks the end-to-end extract with the derived frame loop (frame and frame_plant, both state orders) against the rank-cut loop. Relation, probes, root calls, relation checks and filter passes must all be equal.

All **34** tests of the derived binary pass: the parent's 32 (including round 2's C7 tests and the parent's rank-probe tests) and the 2 new ones (profile/raw/cargo-test-online-rankfinder.txt).

3. Not in scope

- The online target scan (`PIPE_COMPACT`) also canonicalizes with lift_scan. It is 0.33 s of E2E, and a sibling lead is changing the online frame loop, so it is not touched here.
- The remaining key-derivation arithmetic (2 products and 2 `to_normal` maps per candidate) and faster run-finder variants are not part of this exact carry-over.

4. Exactness checks before freezing (cloud-agent VM, correctness only)

profile/exactness_rank.py runs the row's binary and the rank-finder binary as alternating fresh pinned processes, with `arm.py`'s rank-arm environment and no cached table. Per seed it requires the two processes to agree on:

- the rank relation digest and the per-attempt probe vector;
- root calls, total probes, attempts, relations and failures;
- rows without gain;
- every point-addition and frame counter;
- skipped symmetric states;
- the whole index digest, including the layout digests;
- root entries and stored and regular states;
- the target's relation, probes, filter passes and scalar.

It also requires each written factor-log table to equal the base's frozen verified table, and the layout digests to equal the frozen ones. inputs/local_exactness.json records all of it.

Base	Solve	Rank seeds	Identical	Tables equal frozen	Layout equal frozen
M38	frame	1501, 301, 7001, 7002	yes (4/4)	yes	yes
M38	frame_plant	7004, 7005	yes (2/2); 14,077,796 and 12,964,434 keys checked against the rank-cut lift_scan keys	yes	yes
M10	frame	1501, 301, 7003	yes (3/3)	yes	yes
K600	frame	1501, 7006	yes (2/2)	yes	yes

The Q0 identities on all three bases are part of the smoke below and are gated in the run.

5. Frozen design (Tier 1, repeated alternating pairs)

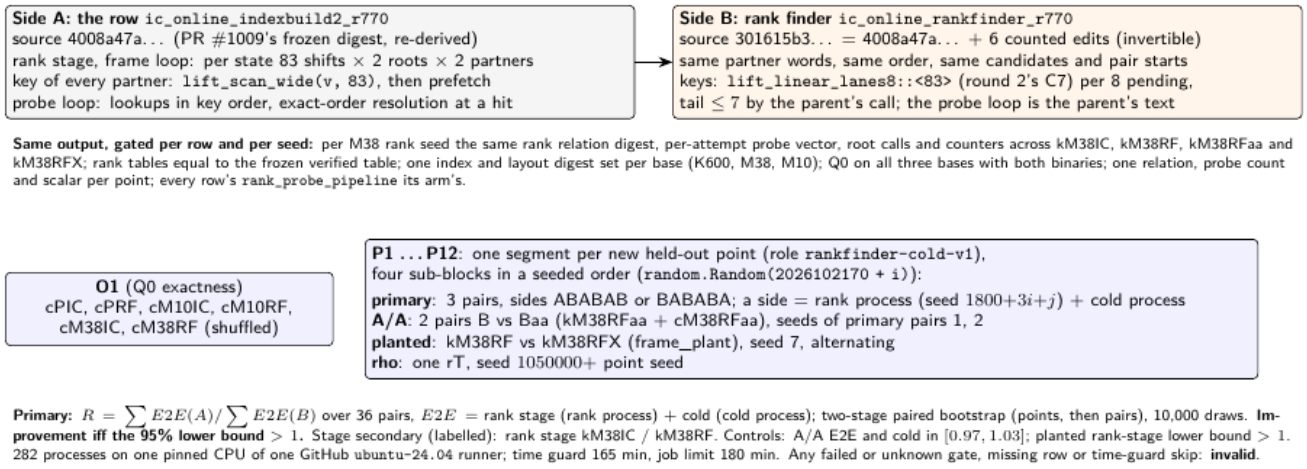


Figure 2: Design (from the frozen protocol and schedule; not a result). Source figures/design.tex, vector figures/design.pdf.

The design is the parent's, itself the Tier 1 template's:

- one GitHub `ubuntu-24.04` runner;
- a fresh process for every run, pinned to one CPU with a clean environment;
- `run_schedule.py`, `arm.py` and `make_schedule.py`, verbatim;
- the two-stage paired bootstrap, the A/A band, and the hardened fail-closed verifier with this experiment's arm names (31 unit tests).

Sides and pairs.

Side	Processes, in order	Binary
A	kM38IC (rank stage, 16 attempts on M38, pair's seed, then the point), then cM38IC (verified table, the point)	the row, `ic_online_indexbuild2_r770` (the parent's arms, verbatim)
B	kM38RF (same seed), then cM38RF	rank finder, `ic_online_rankfinder_r770`
Baa	kM38RFaa, then cM38RFaa	A/A copy of B

- **Primary:** 12 points x **3 pairs = 36 primary pairs**. Each point's six sides strictly alternate, ABABAB or BABABA, with the start drawn per point. Seed of pair j at point i: $1800 + 3i + j$ (1801-1836, new).
- **A/A:** 2 pairs per point (B against Baa), with the seeds of primary pairs 1 and 2.
- **Planted:** one rank pair per point, kM38RF against kM38RFX, alternating. kM38RFX runs `KIC_RANK_SOLVE=frame_plant`, which re-derives every rank key by the rank-cut loop with lift_scan and compares them. It uses `make_schedule.py`'s planted seed 7, the same in every point; that is a control seed, never a primary seed.
- **rho:** one rT per point, seed 1050000 + point seed.
- **O1:** Q0 on cPIC, cPRF, cM10IC, cM10RF, cM38IC and cM38RF.
- In total, **282 processes**. Sub-block order and starts come from `random.Random(2026102170+i)`.

Primary (pre-registered).

- E2E(side) = `timing_ms.rank_stage` (rank process) + `cold_in_process_ms` (cold process), as in the parent.
- R = sum E2E(A) / sum E2E(B) over the 36 pairs.
- 95% two-stage paired bootstrap: points, then pairs within points, with A and B of a pair kept together; 10,000 draws, `random.Random(20261050)`.
- **Improvement iff the lower bound > 1.** Regression iff the upper bound < 1. Otherwise not separated.
- **Invalid** if any gate is false or unknown, if any control fails, or if any row is missing (including time-guard skips).

Stage secondary (labelled): the rank-stage ratio, kM38IC / kM38RF `timing_ms.rank_stage`, with the same bootstrap. The cold processes never run the rank stage, so their ratios (cold, setup, index build, self-test, online) are expected to be about 1 and are reported as controls on binary layout.

Other secondaries (descriptive):

- the geometric mean of the pair ratios;
- CPU time, wall, peak RSS and faults;
- per point, by pair position and by side order;
- the probe law and rank ns per probe;
- S and the gap to rT.

Controls.

- **A/A:** E2E(B)/E2E(Baa) and cold cM38RF/cM38RFaa, each inside [0.97, 1.03] over 24 pairs.
- **Planted timing:** rank stage kM38RFX/kM38RF, lower 95% bound > 1 over 12 pairs.
- **Planted exactness:**
  - every kM38RFX checks > 0 inputs against the rank-cut keys and writes the frozen table (`arm.py`);
  - the closure corruptions are rejected before timing.

Gates. All gates fail closed; an unknown gate counts as a failure:

- the parent's gates (every row ran its arm's binary by name and by digest, one index and layout digest set per base on K600, M38 and M10, Q0 on all six O1 arms, one scalar, relation and probe count per point replayed in the oracle, rank tables equal to the frozen tables, placement, CPU identity, schedule identity and order);
- per rank seed, identical relation digests, probe vectors, root calls and counters across **kM38IC, kM38RF, kM38RFaa and kM38RFX**;
- the probe-law band for kM38IC over its 36 primary seeds (576 attempts);
- **added:** every IC row's `rank_probe_pipeline` is its arm's (absent on side A's binary, `linear_canon_lanes8` on side B's).

Panel and seeds. The panel is 12 new public points, role `rankfinder-cold-v1`, seeds 0..11 (`make_targets.py`). They are checked against every earlier role of the law, including the parent's `indexbuild2-cold-v1`. A role-agnostic x/y scan (`inputs/panel_disjointness.json`) found **0 hits** in all of these:

- origin/main at `d1a10f6d`;
- 861 other origin branches (1,646,556 changed files);
- six sibling scratch worktrees.

The rank seeds 1801-1836 are new (`inputs/seed_disjointness.json`: 86 refs plus the worktrees, 0 hits). The scan's positive control finds the parent's 1501-1536.

Frozen files: `protocol.json` sha256 `3209eee1...`, `schedule.json` sha256 `6810d403...`. `restore.py` checks the following on every build:

- the parent protocol `5cee1a72...`;
- side A's producer equals the parent's frozen `4008a47a...`, and the chain `bc6a31c7...` and `1d328b2e...`;
- both closure headers;
- 30 files copied byte for byte from the parent and 2 from the template;
- every local pin.

6. Power and runtime

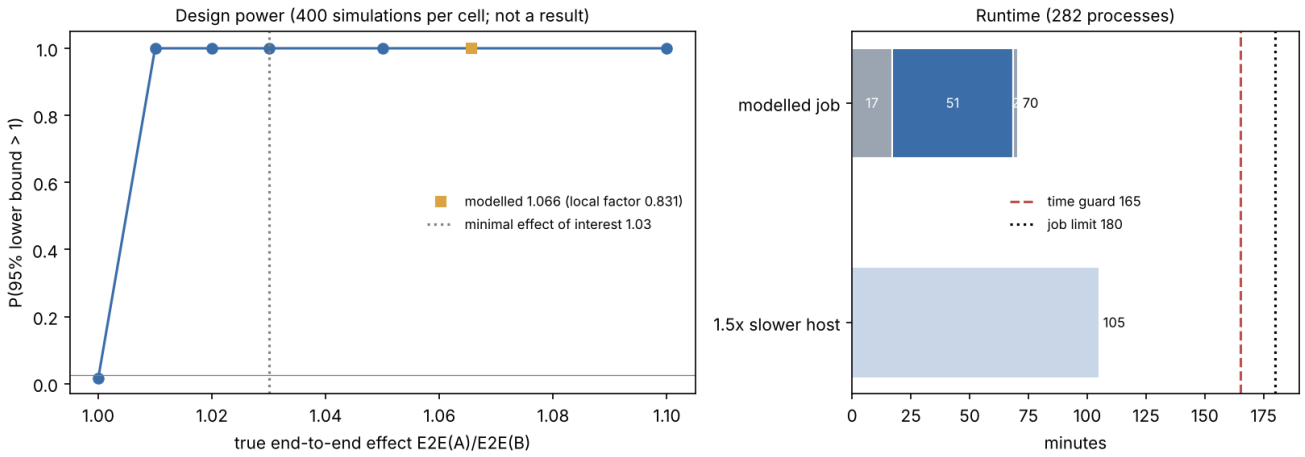


Figure 3: Design-stage power and runtime (not a result), from `analysis/power.json` by `make_figures.py` (svg, pdf).

The prior is the parent's archived run 39464544962 (`make_power_prior.py`, pinned `runs.jsonl` digest `c0d09af5...`):

- the row's 36 primary pairs (kM38IC rank stage; cM38IC cold);
- its 24 A/A ratios, used as per-process noise factors. This is conservative by about sqrt(2).

Side B is modelled as A with the rank stage multiplied by f, and the cold process unchanged. f = 0.831 comes from the local alternating M38 rank processes (four seeds). In the effect cells, f is set to hit each effect. There are 400 simulations per cell and 500 draws each.

True end-to-end effect	P(lower bound > 1)
1.00 (no gain; type I)	0.015
1.01	1.00
1.02	1.00
1.03 (minimal effect of interest)	**1.00**
1.05	1.00
1.10	1.00
modelled, about 1.066	**1.00** (median interval hi/lo 1.011)

At the parent run's means, the model predicts the following. These are estimates for sizing the design, not claims:

- end to end about **1.066** (11.54 s to 10.82 s);
- rank stage about **1.20**.

Round 2's local model overstated its own effect on the runner (modelled 1.16, measured 1.117). This run's verdict rests on the run alone.

The controls behave as follows in simulation:

- the A/A point estimates are inside the band in 100% of simulations;
- the probe-law gate (576 attempts) passes at the archived law in 99.9% and rejects a law 1.5x worse in 100%;
- the planted lower bound exceeds 1 in 100%, modelled at 1.5x. Locally frame_plant is about 2.6x the rank finder's frame loop.

Runtime. The process walls are the parent run's, with side B's rank processes reduced by the modelled saving and the planted arm given 8 s extra. That gives:

- about 253 s per point segment;
- a **51 min** schedule;
- a **job of about 70 min**, with 17 min fixed before and 2 min after;
- **105 min** on a host 1.5x slower.

The time guard is **165 min** and the job limit is **180 min**, so one job fits within the 3 h budget. Any time-guard skip makes the run invalid.

Local smoke of the frozen schedule

See `smoke/README.md`.

Trigger

The run is the workflow `.github/workflows/n83-rankfinder.yml`.

Run

(Recorded after the trigger.)

Files

File	Role
`make_rankfinder_source.py`	the derivation of side B's producer (6 counted, invertible edits on `4008a47a...`)
`protocol.json`, `make_protocol.py`, `schedule.json`	frozen protocol (sha256 `3209eee1...`), its builder, frozen run order (`6810d403...`)
`restore.py`	restore crypto 770f6a69, derive the four producers, regenerate the closure headers, check every parent, template and local pin
`verify_receipt.py`, `test_verify_receipt.py`	the parent's gates, statistics and fail-closed decision rule with this experiment's arms, the planted ratio on the rank stage and the added `rank_probe_pipeline` gate (31 unit tests)
`make_targets.py`, `inputs/cold_targets.json`, `check_panel_disjoint.py`, `inputs/panel_disjointness.json`, `check_seed_disjoint.py`, `inputs/seed_disjointness.json`	the new panel, seeds, and their disjointness records
`profile/`	profiling patch (scratch copies only), runner, exactness runner, summarizer, raw records in `raw/`; `inputs/local_profile.json`, `inputs/local_exactness.json`
`make_power_prior.py`, `inputs/power_prior.json`, `power_check.py`, `analysis/power.json`	power prior (run 39464544962), simulation and result
`arm.py`, `make_schedule.py`, `run_schedule.py`, `check_log_table.py`, `make_*_source.py` (other), `online/Cargo.lock`, `lift2/`, `make_closure_base.py`, `verify_closure.py`, `inputs/*` (parent)	the parent's files, byte for byte
`figures/profile.*`, `figures/power.*`, `make_figures.py`, `figures/design.*`	profile and power figures (SVG, PDF, PNG); design diagram (TikZ source, PDF, PNG)

File	Role
smoke/ README.pdf	local correctness smoke of the frozen schedule this report: pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true